VIGILADA MINEDUCACIÓN

UNIVERSIDAD
DEL QUINDÍO
Res.MEN 014915 - 02 AGO 2022
RENOVACIÓN ACREDITACIÓN

Entrega 2 Proyecto Integrador

Daniela Muñoz Arboleda
María José Marmolejo
Santiago Ovalle Cortes



UNIQUINDÍO
en conexión territorial
www.uniquindio.edu.co

Prueba 0 · Paquete y estado de los objetos (paquete, 10 %)

PKG_RESERVAS agrupa la función, el procedimiento de reserva y las dos liquidaciones. En el cuerpo quedan privados el cursor c_reservas y el procedimiento prc_rango_mes, que comparten las dos liquidaciones. Esta prueba muestra que todo se compiló y qué expone el paquete.

Comprobación	Esperado	Obtenido
PACKAGE y PACKAGE BODY PKG_RESERVAS	VALID	VALID
Subprogramas públicos	4	FN_VALOR_ESTADIA, SP_CREAR_RESERVA, SP_LIQUIDACION_MASIVA, SP_LIQUIDACION_MENSUAL
user_errors	Sin filas	Sin filas
Disparadores	2, COMPOUND, ENABLED	TRG_RH_SIN_SOLAPE y TRG_TARIFA_AUDITORIA, COMPOUND, ENABLED
Estados de la carga	Los 4	Completada 20.081, Cancelada 3.654, Confirmada 1.229, Pendiente 536
Volumen de la carga	Todo sobre el mínimo, 0 solapes	60 alojamientos, 708 habitaciones, 3.000 clientes, 25.500 reservas, 33.689 líneas, 36.329 pagos, 45.182 servicios contratados, reseñas en 59,9 % de las completadas, 0 solapes

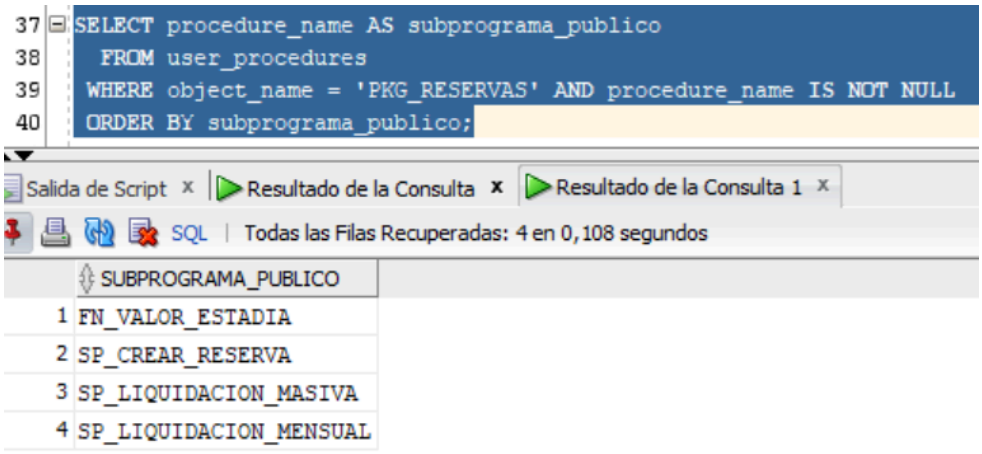
Figura 1. ***Objetos válidos del esquema y subprogramas públicos de PKG_RESERVAS***

```
31  SELECT object_name, object_type, status
32    FROM user_objects
33  WHERE object_name IN ('PKG_RESERVAS', 'TRG_TARIFA_AUDITORIA', 'TRG_RH_SIN_SOLAPE'
34                        'MV_OCUPACION_MENSUAL', 'RESERVA_HISTORICO')
35  ORDER BY object_name, object_type;
```

Salida de Script | Resultado de la Consulta | Resultado de la Consulta 1

SQL | Todas las Filas Recuperadas: 7 en 0,035 segundos

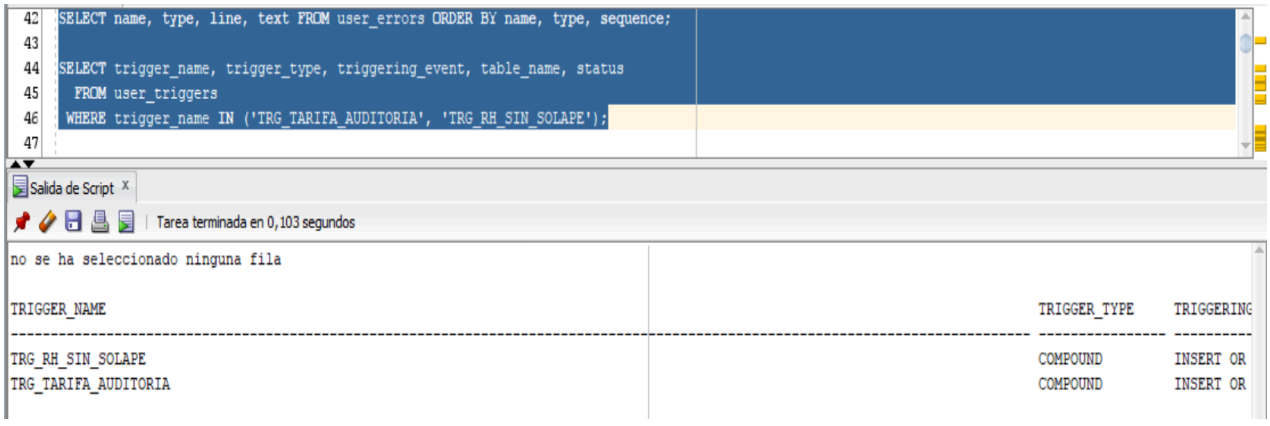
	OBJECT_NAME	OBJECT_TYPE	STATUS
1	MV_OCUPACION_MENSUAL	MATERIALIZED VIEW	VALID
2	MV_OCUPACION_MENSUAL	TABLE	VALID
3	PKG_RESERVAS	PACKAGE	VALID
4	PKG_RESERVAS	PACKAGE BODY	VALID
5	RESERVA_HISTORICO	TABLE	VALID
6	TRG_RH_SIN_SOLAPE	TRIGGER	VALID
7	TRG_TARIFA_AUDITORIA	TRIGGER	VALID



Fuente: elaboración propia en Oracle SQL Developer

Las dos consultas al diccionario de datos muestran que el paquete, su cuerpo, los dos disparadores, la tabla del histórico y la vista quedaron compilados y en estado VALID, y que el paquete expone cuatro subprogramas públicos. El cursor c_reservas y el procedimiento prc_rango_mes no aparecen porque se declararon solo en el cuerpo: son privados, ya que únicamente los usan las liquidaciones.

Figura 2. ***Errores de compilación y disparadores del esquema***



Fuente: elaboración propia en Oracle SQL Developer.

Que user_errors no devuelva filas demuestra que ningún objeto tiene errores de compilación. Los dos disparadores son COMPOUND porque necesitan actuar en dos momentos, por cada fila y al final de la sentencia (ver Justificación técnica), y ambos están habilitados sobre la tabla que protegen.

Figura 3. ***Reservas de la carga por estado***

```
48  -- Estados que usa la carga (para leer bien las demas pruebas)
49  SELECT estado, COUNT(*) AS reservas FROM reserva GROUP BY estado ORDER BY estado;
```

Salida de Script | Resultado de la Consulta

SQL | Todas las Filas Recuperadas: 4 en 0,096 segundos

	ESTADO	RESERVAS
1	Cancelada	3654
2	Completada	20081
3	Confirmada	1229
4	Pendiente	536

Fuente: elaboración propia en Oracle SQL Developer.

La carga tiene reservas en los cuatro estados del enunciado. Esto es necesario para probar las reglas que dependen del estado: las canceladas no bloquean habitaciones ni cuentan como ingreso, y solo Confirmada y Completada entran en la liquidación y en la ocupación.

Figura 4. ***Volumen y calidad de la carga de datos***

```
620  -- Volumen por tabla frente al minimo del enunciado
621  SELECT 'municipio' AS tabla, COUNT(*) AS filas, 12 AS minimo FROM municipio
622  UNION ALL SELECT 'tipo_alojamiento',    COUNT(*), 4     FROM tipo_alojamiento
623  UNION ALL SELECT 'alojamiento',         COUNT(*), 60    FROM alojamiento
```

Salida de Script | Tarea terminada en 1,427 segundos

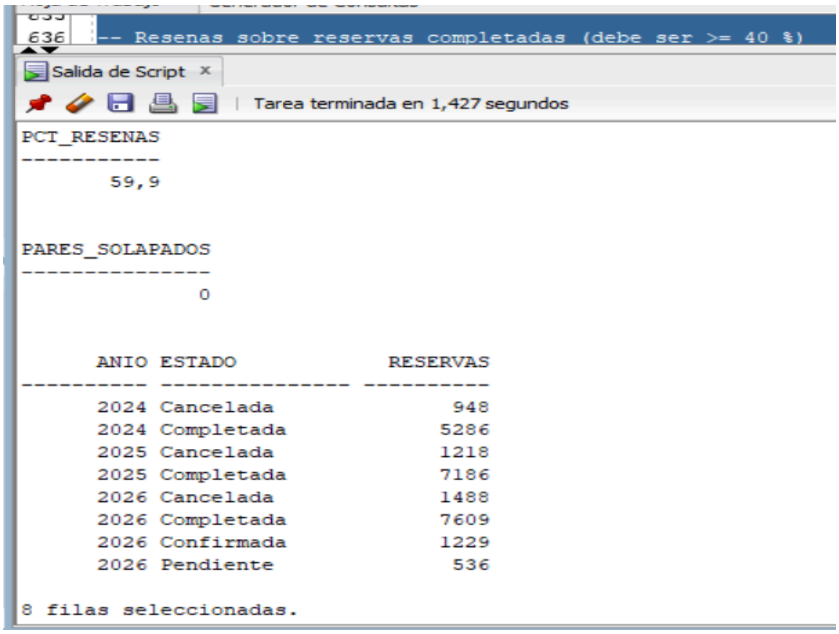
```
TABLA                FILAS     MINIMO
-------------------- ---------- ----------
municipio                12         12
tipo_alojamiento          5          4
alojamiento              60         60
habitacion              708        400
temporada                22          6
tarifa                15576
cliente                3000       3000
reserva               25500      25000
reserva_habitacion    33689      25000
pago                  36329      25000
servicio                203         30
```

```
619
620  -- Volumen por tabla frente al minimo del enunciado
621  SELECT 'municipio' AS tabla, COUNT(*) AS filas, 12 AS minimo FROM municipio
622  UNION ALL SELECT 'tipo_alojamiento',    COUNT(*), 4     FROM tipo_alojamiento
623  UNION ALL SELECT 'alojamiento',         COUNT(*), 60    FROM alojamiento
```

Salida de Script | Tarea terminada en 1,427 segundos

```
TABLA                FILAS     MINIMO
-------------------- ---------- ----------
reserva_servicio      45182      40000
resena                12020
usuario_sistema          63         10

14 filas seleccionadas.
```



```
636  -- Resenas sobre reservas completadas (debe ser >= 40 %)
```

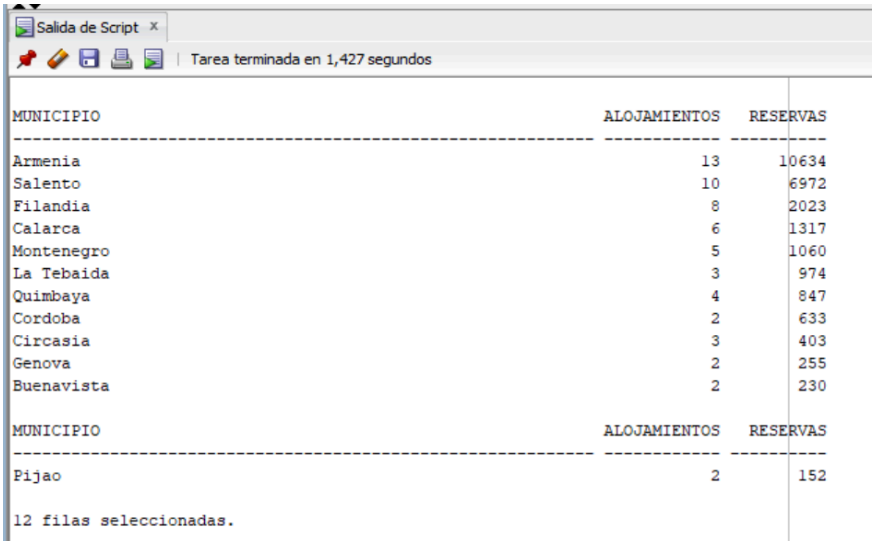
Salida de Script | Tarea terminada en 1,427 segundos

```
PCT_RESENAS
-----------
       59,9


PARES_SOLAPADOS
---------------
              0


      ANIO ESTADO            RESERVAS
---------- --------------- ----------
      2024 Cancelada              948
      2024 Completada            5286
      2025 Cancelada             1218
      2025 Completada            7186
      2026 Cancelada             1488
      2026 Completada            7609
      2026 Confirmada            1229
      2026 Pendiente              536

8 filas seleccionadas.
```



Salida de Script | Tarea terminada en 1,427 segundos

```
MUNICIPIO                                                  ALOJAMIENTOS   RESERVAS
---------------------------------------------------------- ------------ ----------
Armenia                                                              13      10634
Salento                                                              10       6972
Filandia                                                              8       2023
Calarca                                                               6       1317
Montenegro                                                            5       1060
La Tebaida                                                            3        974
Quimbaya                                                              4        847
Cordoba                                                               2        633
Circasia                                                              3        403
Genova                                                                2        255
Buenavista                                                            2        230

MUNICIPIO                                                  ALOJAMIENTOS   RESERVAS
---------------------------------------------------------- ------------ ----------
Pijao                                                                 2        152

12 filas seleccionadas.
```

Fuente: elaboración propia en Oracle SQL Developer.

La carga corregida cumple todos los mínimos del enunciado y no tiene habitaciones reservadas dos veces en la misma noche, que eran los problemas señalados en la Entrega 1. Se generó con semilla fija para que cualquier persona reproduzca exactamente estos datos y estos resultados.

Prueba 1 · fn_valor_estadia (20 %)

La función genera una fila por noche (check-in incluido, check-out excluido), cruza cada noche con su temporada y suma la tarifa de la habitación en esa temporada. Si una noche no tiene tarifa lanza -20002 en vez de devolver un valor incompleto; si una noche cae en dos temporadas lanza -20012 en vez de cobrar dos veces.

Caso: habitación 1, del 9 al 15 de octubre de 2026, 6 noches que cruzan tres temporadas.

Noche	Temporada	Precio (COP)
2026-10-09	Media agosto-octubre 2026	235.000
2026-10-10	Media agosto-octubre 2026	235.000
2026-10-11	Puente festivo octubre 2026 (Alta)	352.000
2026-10-12	Puente festivo octubre 2026 (Alta)	352.000
2026-10-13	Puente festivo octubre 2026 (Alta)	352.000
2026-10-14	Baja octubre-noviembre 2026	148.000
Total	2 × 235.000 + 3 × 352.000 + 148.000	1.674.000

Comprobación	Esperado	Obtenido
suma_a_mano	Igual a valor_funcion	1.674.000
valor_funcion	Igual a suma_a_mano	1.674.000
Salida igual a entrada	ORA-20001	ORA-20001
Fechas sin temporada (2030)	ORA-20002	ORA-20002: sin tarifa para 2 de las 2 noches
Habitación 99999	ORA-20003	ORA-20003

Figura 5. ***Valor noche por noche de la estadía del 9 al 15 de octubre de 2026***

```
-- 1.a Detalle noche por noche, calculado a mano (sin la funcion)
SELECT n.noche, te.nombre AS temporada, te.tipo, t.precio_noche
  FROM (SELECT DATE '2026-10-09' + LEVEL - 1 AS noche
          FROM dual CONNECT BY LEVEL <= 6) n
  JOIN temporada te ON n.noche BETWEEN te.fecha_inicio AND te.fecha_fin
  JOIN tarifa t     ON t.id_temporada = te.id_temporada AND t.id_habitacion = 1
 ORDER BY n.noche;
```

Salida de Script | Resultado de la Consulta

SQL | Todas las Filas Recuperadas: 6 en 0,057 segundos

	NOCHE	TEMPORADA	TIPO	PRECIO_NOCHE
1	09/10/26	Media agosto-octubre 2026	Media	235000
2	10/10/26	Media agosto-octubre 2026	Media	235000
3	11/10/26	Puente festivo octubre 2026	Alta	352000
4	12/10/26	Puente festivo octubre 2026	Alta	352000
5	13/10/26	Puente festivo octubre 2026	Alta	352000
6	14/10/26	Baja octubre-noviembre 2026	Baja	148000

Fuente: elaboración propia en Oracle SQL Developer.

La consulta calcula a mano el precio de cada noche de la estadía. Se eligió una estadía que cruza tres temporadas (media, puente festivo y baja) porque es justo el caso que pide resolver el enunciado. El 15 de octubre no aparece porque el día de salida no se cobra.

Figura 6. ***Comparación entre la suma a mano y fn_valor_estadia***

```
-- 1.b Suma a mano frente a la funcion
SELECT (SELECT SUM(t.precio_noche)
          FROM (SELECT DATE '2026-10-09' + LEVEL - 1 AS noche
                  FROM dual CONNECT BY LEVEL <= 6) n
          JOIN temporada te ON n.noche BETWEEN te.fecha_inicio AND te.fecha_fin
          JOIN tarifa t     ON t.id_temporada = te.id_temporada AND t.id_habitacion = 1) AS suma_a_mano,
       pkg_reservas.fn_valor_estadia(1, DATE '2026-10-09', DATE '2026-10-15') AS valor_funcion
  FROM dual;
```

Salida de Script | Resultado de la Consulta

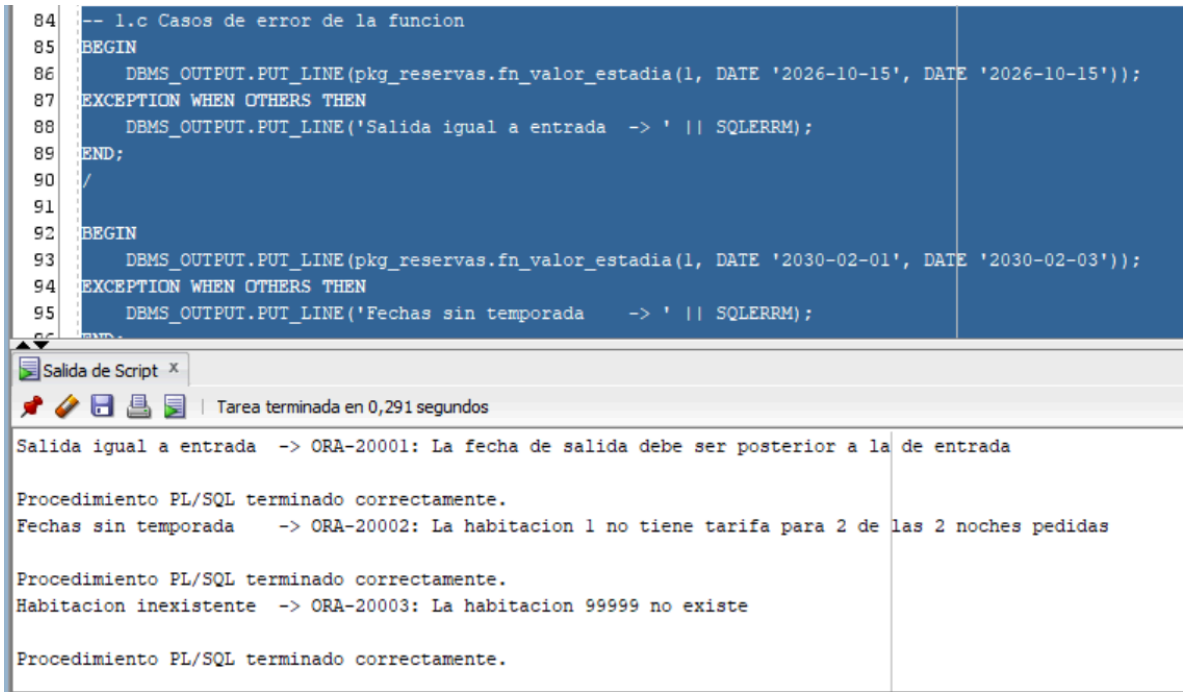
SQL | Todas las Filas Recuperadas: 1 en 0,036 segundos

	SUMA_A_MANO	VALOR_FUNCION
1	1674000	1674000

Fuente: elaboración propia en Oracle SQL Developer.

La suma a mano y el resultado de fn_valor_estadia coinciden en 1.674.000. La función aplica a cada noche la tarifa de su temporada; multiplicar una sola tarifa por el número de noches daría un valor incorrecto en las estadías que cruzan temporadas.

Figura 7. ***Errores propios de fn_valor_estadia***



Fuente: elaboración propia en Oracle SQL Developer.

La función rechaza con errores propios los datos que darían un valor incorrecto: fechas invertidas o iguales, noches sin tarifa y habitaciones inexistentes. Se prefirió lanzar un error antes que devolver un valor incompleto que pudiera cobrarse mal.

Prueba 2 · sp_crear_reserva (15 %)

El procedimiento valida, en este orden, fechas, huéspedes, habitación, cliente, capacidad, disponibilidad y tarifas. Solo si todo pasa inserta la reserva en estado Pendiente y su línea en RESERVA_HABITACION. No hace COMMIT: lo decide quien lo llama. La prueba usa enero de 2027, donde la carga no tiene reservas, y termina en ROLLBACK.

Caso	Esperado	Obtenido
1. Reserva válida del 5 al 8 de enero de 2027	Crea la reserva	Reserva 25501 en la habitación 1 (capacidad 2), valor 1.011.000
2. Misma habitación del 6 al 9	ORA-20006	ORA-20006
3. Salida antes de la entrada	ORA-20001	ORA-20001

Caso	Esperado	Obtenido
4. Salida igual a la entrada	ORA-20001	ORA-20001
5. Capacidad máxima + 1 huésped	ORA-20005	ORA-20005: máximo 2, se pidieron 3
6. Cero huéspedes	ORA-20007	ORA-20007
7. Habitación 99999	ORA-20003	ORA-20003
8. Cliente 99999	ORA-20004	ORA-20004
9. Se cancela la 1 y se repite el caso 2	Crea la reserva	Reserva 25502 en las mismas fechas

Figura 8. ***Ejecución de sp_crear_reserva: caso válido y validaciones***

```
113 DECLARE
114     v_hab  habitacion.id_habitacion%TYPE;
115     v_cap  habitacion.capacidad_maxima%TYPE;
116     v_res  reserva.id_reserva%TYPE;
117     v_res1 reserva.id_reserva%TYPE;
118 BEGIN
119     SELECT id_habitacion, capacidad_maxima INTO v_hab, v_cap
120       FROM (SELECT id_habitacion, capacidad_maxima FROM habitacion
```

Salida de Script

Tarea terminada en 0,342 segundos

```
Habitacion de prueba: 1 (capacidad 2)
Reserva 25501 creada. Valor de la estadia: 1011000
1) Reserva valida -> id 25501
2) Habitacion ocupada    -> ORA-20006: La habitacion 1 no esta disponible entre 2027-01-06 y 2027-01-09
3) Salida antes          -> ORA-20001: La fecha de salida debe ser posterior a la de entrada
4) Salida igual          -> ORA-20001: La fecha de salida debe ser posterior a la de entrada
5) Capacidad excedida    -> ORA-20005: La habitacion 1 admite maximo 2 huespedes y se pidieron 3
6) Sin huespedes         -> ORA-20007: La reserva debe tener al menos un huesped
7) Habitacion inexistente-> ORA-20003: La habitacion 99999 no existe
8) Cliente inexistente   -> ORA-20004: El cliente 99999 no existe
Reserva 25502 creada. Valor de la estadia: 1011000
9) Tras cancelar la reserva 25501, las mismas fechas se reservaron en la 25502

Procedimiento PL/SQL terminado correctamente.
```

Fuente: elaboración propia en Oracle SQL Developer.

El procedimiento crea la reserva solo cuando todas las validaciones pasan y lanza un error propio para cada caso inválido, incluidos los tres que pide el enunciado: habitación no disponible (-20006), salida anterior o igual a la entrada (-20001) y capacidad excedida (-20005). El caso 9 muestra que una reserva cancelada libera la habitación. La prueba usa enero de 2027 y termina en ROLLBACK para no dejar datos.

Figura 9. ***Excepciones propias de sp_crear_reserva en el código***

```
    BEGIN
        IF p_checkin IS NULL OR p_checkout IS NULL OR v_fin <= v_ini THEN
            RAISE_APPLICATION_ERROR(-20001,
                'La fecha de salida debe ser posterior a la de entrada');
        END IF;
--
        IF p_huespedes IS NULL OR p_huespedes < 1 THEN
            RAISE_APPLICATION_ERROR(-20007,
                'La reserva debe tener al menos un huesped');
        END IF;
--
        BEGIN
            SELECT capacidad_maxima, id_alojamiento
              INTO v_capacidad, v_id_alojamiento
              FROM habitacion
             WHERE id_habitacion = p_id_habitacion;
        EXCEPTION
            WHEN NO_DATA_FOUND THEN
                RAISE_APPLICATION_ERROR(-20003,
                    'La habitacion ' || p_id_habitacion || ' no existe');
        END;
--
```

```
        SELECT COUNT(*) INTO v_clientes
          FROM cliente
         WHERE id_cliente = p_id_cliente;
        IF v_clientes = 0 THEN
            RAISE_APPLICATION_ERROR(-20004,
                'El cliente ' || p_id_cliente || ' no existe');
        END IF;
--
        IF p_huespedes > v_capacidad THEN
            RAISE_APPLICATION_ERROR(-20005,
                'La habitacion ' || p_id_habitacion || ' admite maximo '
                || v_capacidad || ' huespedes y se pidieron ' || p_huespedes);
        END IF;
--
        SELECT COUNT(*) INTO v_cruces
          FROM reserva_habitacion rh
          JOIN reserva r ON r.id_reserva = rh.id_reserva
         WHERE rh.id_habitacion = p_id_habitacion
           AND UPPER(r.estado) <> 'CANCELADA'
           AND rh.fecha_checkin  < v_fin
           AND rh.fecha_checkout > v_ini;
        IF v_cruces > 0 THEN
            RAISE_APPLICATION_ERROR(-20006,
                'La habitacion ' || p_id_habitacion
                || ' no esta disponible entre ' || TO_CHAR(v_ini, 'YYYY-MM-DD')
                || ' y ' || TO_CHAR(v_fin, 'YYYY-MM-DD'));
        END IF;
```

Fuente: elaboración propia en Oracle SQL Developer.

Las validaciones se hacen antes de cualquier INSERT, de modo que una reserva inválida nunca queda escrita a medias. Cada error tiene un código fijo entre -20001 y -20007, lo que le permite a quien llame al procedimiento saber exactamente qué falló.

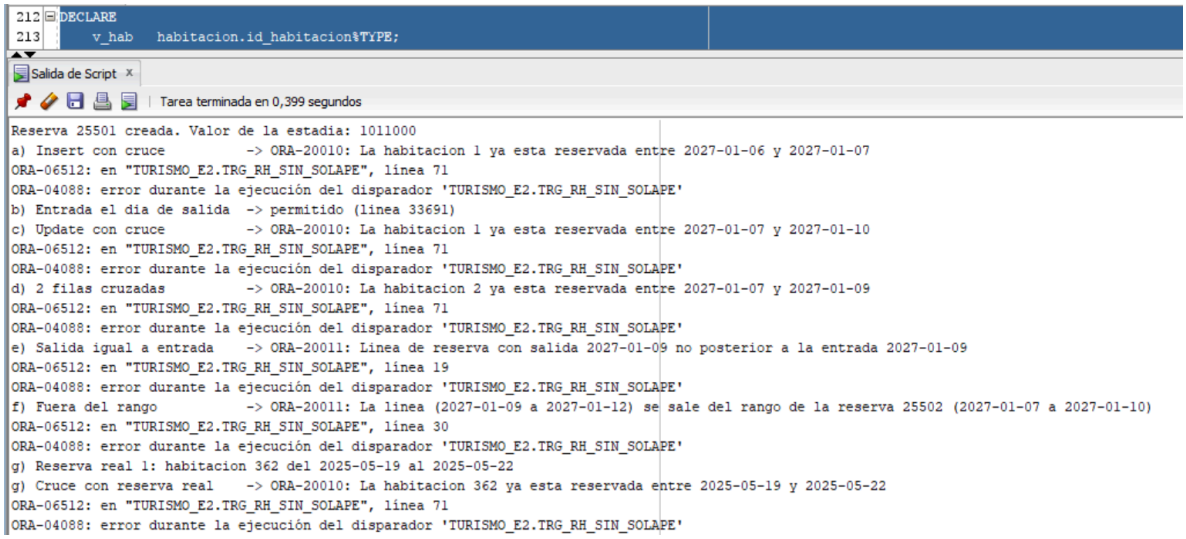
Prueba 3 · Disparador de fila: solape de reservas (10 %)

TRG_RH_SIN_SOLAPE valida cada fila en su sección BEFORE EACH ROW (salida posterior a la entrada y línea dentro del rango de su reserva) y, al cerrar la sentencia, compara cada fila con el resto de la tabla. Hay cruce cuando la entrada nueva es menor que la salida de la otra y la salida nueva es mayor que su entrada; las reservas canceladas no se bloquean. Los inserts y updates van directo a la tabla, sin el procedimiento, para demostrar que la tabla se protege sola.

Caso	Esperado	Obtenido
a. Insert que se cruza con la reserva del 5 al 8 de enero	ORA-20010	ORA-20010: habitación 1, 6 al 7 de enero
b. Entrar el 8, el día en que sale la otra	Permitido	Permitido (línea 33691)
c. UPDATE que adelanta esa línea al día 7	ORA-20010	ORA-20010: 7 al 10 de enero
d. Un solo INSERT con 2 filas cruzadas entre sí	ORA-20010	ORA-20010: habitación 2, 7 al 9 de enero
e. Línea con salida igual a la entrada	ORA-20011	ORA-20011
f. Línea por fuera del rango de su reserva	ORA-20011	ORA-20011: 9 al 12 se sale de 7 al 10
g. Cruce con una reserva real de la carga	ORA-20010	ORA-20010: reserva 1, habitación 362, 19 al 22 de mayo de 2025

Los casos c y d explican por qué el disparador es compuesto: con un disparador de fila simple que consulta su propia tabla, un UPDATE o un INSERT de varias filas da ORA-04091 (tabla mutante). Debajo de cada error, Oracle agrega ORA-06512 y ORA-04088 con la línea de TRG_RH_SIN_SOLAPE que lo lanzó: es la prueba de que el rechazo viene del disparador y no del procedimiento.

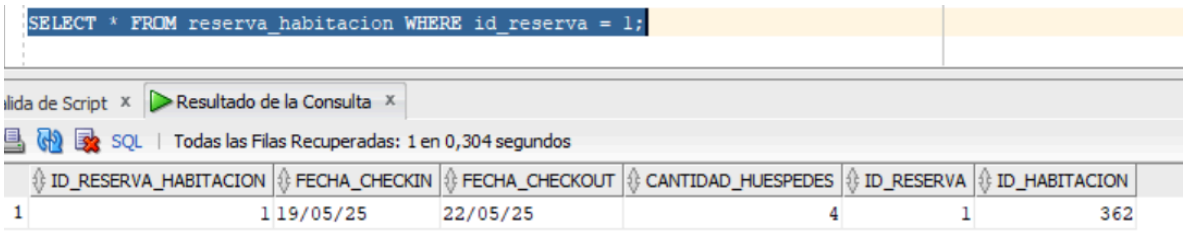
Figura 10. ***Casos del disparador de fila TRG_RH_SIN_SOLAPE***



Fuente: elaboración propia en Oracle SQL Developer.

Los inserts y updates van directo a RESERVA_HABITACION, sin pasar por el procedimiento, para demostrar que la tabla se protege sola. Se rechazan los cruces (casos a, c, d y g) y las líneas con fechas inválidas (e y f), y se permite entrar el mismo día en que sale otra reserva (b). Las líneas ORA-06512 y ORA-04088 confirman que el rechazo lo lanzó el disparador.

Figura 11. ***Reserva real de la carga usada en el caso g***



Fuente: elaboración propia en Oracle SQL Developer.

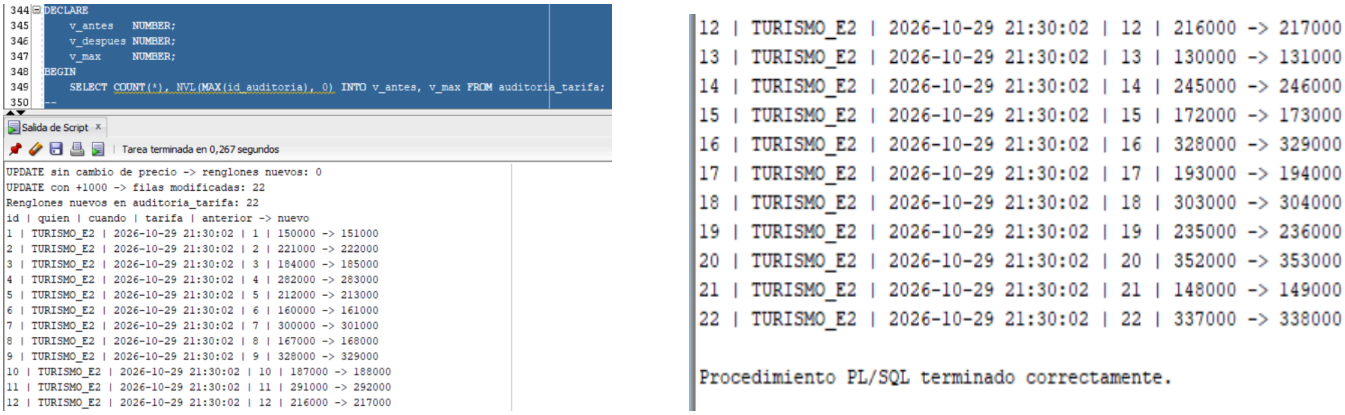
El caso g usa una reserva real de la carga y no una creada para la prueba, como pide la rúbrica («probado con un caso real»). La consulta confirma que esa reserva existe con la misma habitación y las mismas fechas que el disparador protegió.

Prueba 4 · Disparador de sentencia: auditoría de TARIFA (10 %)

TRG_TARIFA_AUDITORIA anota cada fila modificada en una colección y, en su sección AFTER STATEMENT, escribe todos los renglones de una vez en AUDITORIA_TARIFA: quién (usuario de la sesión), cuándo (SYSDATE, igual para toda la sentencia), precio anterior y precio nuevo. Un UPDATE que deja el precio igual no audita nada. La prueba termina en ROLLBACK.

Comprobación	Esperado	Obtenido
Renglones nuevos con UPDATE sin cambio de precio	0	0
Filas modificadas por UPDATE +1000	22 (una por temporada)	22
Renglones nuevos en auditoria_tarifa	Igual a las filas modificadas	22
Quién y cuándo	Usuario de la sesión, misma fecha y hora en todos	TURISMO_E2, la misma fecha y hora en los 22
precio_nuevo	precio_anterior + 1000	Sí; por ejemplo, tarifa 1: 150.000 → 151.000

Figura 12. ***Auditoría generada por un UPDATE sobre TARIFA***



```
DECLARE
    v_antes   NUMBER;
    v_despues NUMBER;
    v_max     NUMBER;
BEGIN
    SELECT COUNT(*), NVL(MAX(id_auditoria), 0) INTO v_antes, v_max FROM auditoria_tarifa;
--
Salida de Script
Tarea terminada en 0,267 segundos
UPDATE sin cambio de precio -> renglones nuevos: 0
UPDATE con +1000 -> filas modificadas: 22
Renglones nuevos en auditoria_tarifa: 22
id | quien | cuando | tarifa | anterior -> nuevo
1 | TURISMO_E2 | 2026-10-29 21:30:02 | 1 | 150000 -> 151000
2 | TURISMO_E2 | 2026-10-29 21:30:02 | 2 | 221000 -> 222000
3 | TURISMO_E2 | 2026-10-29 21:30:02 | 3 | 184000 -> 185000
4 | TURISMO_E2 | 2026-10-29 21:30:02 | 4 | 282000 -> 283000
5 | TURISMO_E2 | 2026-10-29 21:30:02 | 5 | 212000 -> 213000
6 | TURISMO_E2 | 2026-10-29 21:30:02 | 6 | 160000 -> 161000
7 | TURISMO_E2 | 2026-10-29 21:30:02 | 7 | 300000 -> 301000
8 | TURISMO_E2 | 2026-10-29 21:30:02 | 8 | 167000 -> 168000
9 | TURISMO_E2 | 2026-10-29 21:30:02 | 9 | 328000 -> 329000
10 | TURISMO_E2 | 2026-10-29 21:30:02 | 10 | 187000 -> 188000
11 | TURISMO_E2 | 2026-10-29 21:30:02 | 11 | 291000 -> 292000
12 | TURISMO_E2 | 2026-10-29 21:30:02 | 12 | 216000 -> 217000
12 | TURISMO_E2 | 2026-10-29 21:30:02 | 12 | 216000 -> 217000
13 | TURISMO_E2 | 2026-10-29 21:30:02 | 13 | 130000 -> 131000
14 | TURISMO_E2 | 2026-10-29 21:30:02 | 14 | 245000 -> 246000
15 | TURISMO_E2 | 2026-10-29 21:30:02 | 15 | 172000 -> 173000
16 | TURISMO_E2 | 2026-10-29 21:30:02 | 16 | 328000 -> 329000
17 | TURISMO_E2 | 2026-10-29 21:30:02 | 17 | 193000 -> 194000
18 | TURISMO_E2 | 2026-10-29 21:30:02 | 18 | 303000 -> 304000
19 | TURISMO_E2 | 2026-10-29 21:30:02 | 19 | 235000 -> 236000
20 | TURISMO_E2 | 2026-10-29 21:30:02 | 20 | 352000 -> 353000
21 | TURISMO_E2 | 2026-10-29 21:30:02 | 21 | 148000 -> 149000
22 | TURISMO_E2 | 2026-10-29 21:30:02 | 22 | 337000 -> 338000

Procedimiento PL/SQL terminado correctamente.
```

Fuente: elaboración propia en Oracle SQL Developer.

Un solo UPDATE sobre 22 tarifas genera 22 renglones de auditoría con quién (TURISMO_E2), cuándo (la misma fecha y hora para toda la sentencia), precio anterior y precio nuevo, que es lo que pide el enunciado. El UPDATE que no cambia el precio no audita nada, para no llenar la tabla con registros sin información.

Figura 13. ***Código del disparador de sentencia TRG_TARIFA_AUDITORIA***

```
CREATE OR REPLACE TRIGGER trg_tarifa_auditoria
FOR INSERT OR UPDATE OF precio_noche ON tarifa
COMPOUND TRIGGER
--
    TYPE t_cambio  IS RECORD (
        id_tarifa tarifa.id_tarifa%TYPE,
        anterior  tarifa.precio_noche%TYPE,
        nuevo     tarifa.precio_noche%TYPE
    );
    TYPE t_cambios IS TABLE OF t_cambio INDEX BY PLS_INTEGER;
    g_cambios t_cambios;
--
    -- Por cada fila afectada solo se anota el cambio
    AFTER EACH ROW IS
        v_n PLS_INTEGER;
    BEGIN
        IF INSERTING OR :OLD.precio_noche <> :NEW.precio_noche THEN
            v_n := g_cambios.COUNT + 1;
            g_cambios(v_n).id_tarifa := :NEW.id_tarifa;
            g_cambios(v_n).anterior  := :OLD.precio_noche;  -- NULL si es insert
            g_cambios(v_n).nuevo     := :NEW.precio_noche;
        END IF;
    END AFTER EACH ROW;
    -- Al terminar la sentencia se escribe todo de una vez
    AFTER STATEMENT IS
        v_base  NUMBER;
        v_ahora DATE := SYSDATE;
    BEGIN
        IF g_cambios.COUNT > 0 THEN
            SELECT NVL(MAX(id_auditoria), 0) INTO v_base FROM auditoria_tarifa;
            FOR i IN 1 .. g_cambios.COUNT LOOP
                INSERT INTO auditoria_tarifa (id_auditoria, usuario_bd,
                       fecha_cambio, precio_anterior, precio_nuevo, id_tarifa)
                VALUES (v_base + i,
                        SYS_CONTEXT('USERENV', 'SESSION_USER'),
                        v_ahora,
                        g_cambios(i).anterior,
                        g_cambios(i).nuevo,
                        g_cambios(i).id_tarifa);
            END LOOP;
        END IF;
    END AFTER STATEMENT;
--
END trg_tarifa_auditoria;
/
```

Fuente: elaboración propia en Oracle SQL Developer.

La sección AFTER EACH ROW solo guarda cada cambio en una colección, porque es el único momento en que se conocen :OLD y :NEW. La sección AFTER STATEMENT escribe toda la auditoría una vez, al final de la sentencia, que es el comportamiento de disparador de sentencia que pide el enunciado.

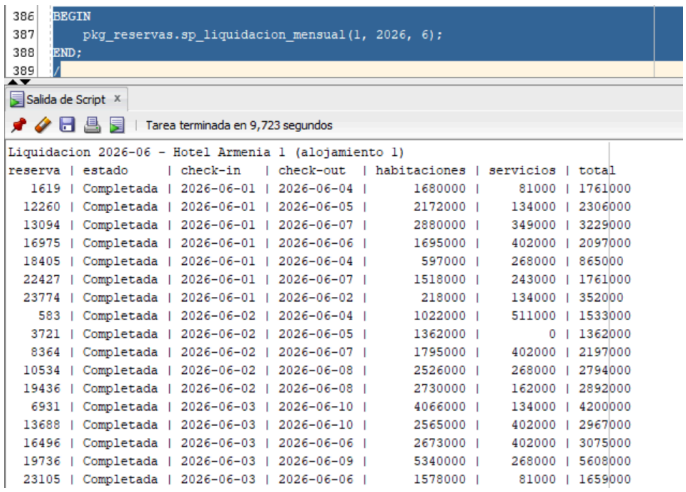
Prueba 5 · Cursor de liquidación mensual (15 %)

El cursor explícito c_reservas(alojamiento, desde, hasta) trae las reservas Confirmada y Completada con check-in en el mes, con el valor de sus habitaciones (noche por noche con fn_valor_estadia) y de sus servicios. sp_liquidacion_mensual lo recorre con OPEN, FETCH y CLOSE para un alojamiento; sp_liquidacion_masiva lo abre con parámetros distintos para cada alojamiento. El resultado se comprueba con consultas aparte que no usan ni el cursor ni la función.

Comprobación (alojamiento 1, junio 2026)	Valor (COP)
Reservas liquidadas (5.a)	119
Reservas en la verificación (5.b)	119
Habitaciones (5.a)	229.531.000
Habitaciones en la verificación (5.b)	229.531.000
Servicios (5.a)	31.247.000
Servicios en la verificación (5.c)	31.247.000
TOTAL A LIQUIDAR (5.a)	260.778.000
Total del alojamiento 1 en la liquidación masiva (5.d)	260.778.000
Liquidación masiva de junio 2026 (5.d)	60 alojamientos, 1.371 reservas, 2.679.444.000
Mes 13	ORA-20008
Alojamiento 999	ORA-20009

Cada pareja de filas debe coincidir. TOTAL A LIQUIDAR = habitaciones + servicios.

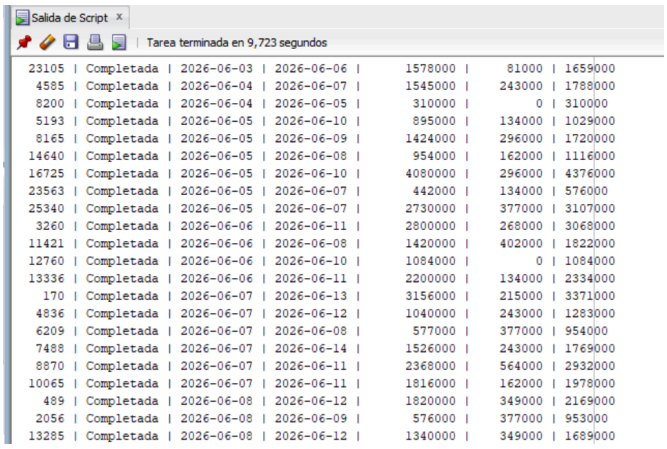
Figura 14. ***Liquidación mensual del alojamiento 1, junio de 2026***



```
BEGIN
    pkg_reservas.sp_liquidacion_mensual(1, 2026, 6);
END;
/
```

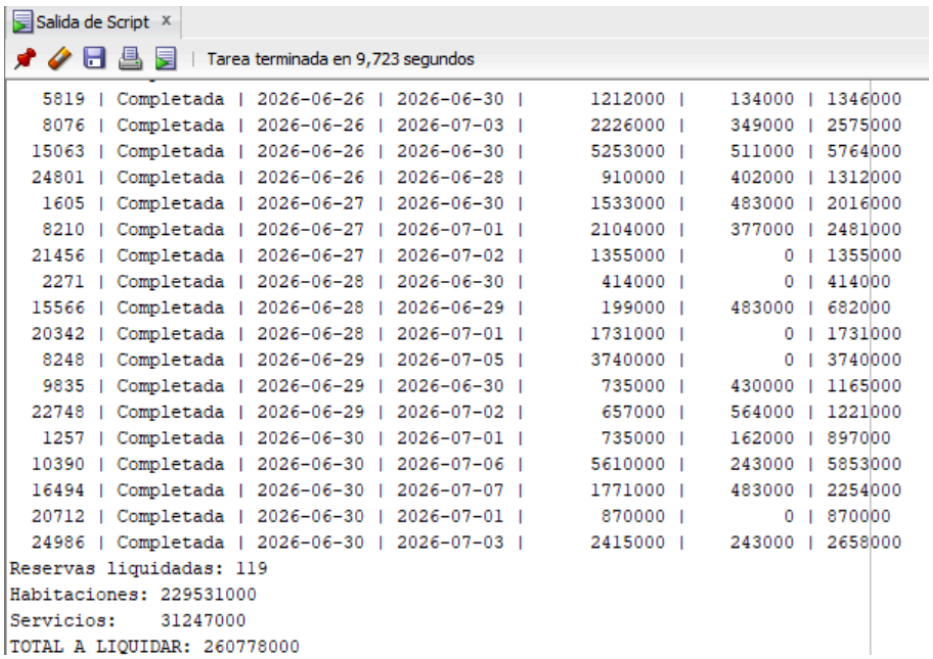
Salida de Script | Tarea terminada en 9,723 segundos

```
Liquidacion 2026-06 - Hotel Armenia 1 (alojamiento 1)
reserva | estado     | check-in   | check-out  | habitaciones | servicios | total
   1619 | Completada | 2026-06-01 | 2026-06-04 |      1680000 |     81000 | 1761000
  12260 | Completada | 2026-06-01 | 2026-06-05 |      2172000 |    134000 | 2306000
  13094 | Completada | 2026-06-01 | 2026-06-07 |      2880000 |    349000 | 3229000
  16975 | Completada | 2026-06-01 | 2026-06-06 |      1695000 |    402000 | 2097000
  18405 | Completada | 2026-06-01 | 2026-06-04 |       597000 |    268000 | 865000
  22427 | Completada | 2026-06-01 | 2026-06-07 |      1518000 |    243000 | 1761000
  23774 | Completada | 2026-06-01 | 2026-06-02 |       218000 |    134000 | 352000
    583 | Completada | 2026-06-02 | 2026-06-04 |      1022000 |    511000 | 1533000
   3721 | Completada | 2026-06-02 | 2026-06-05 |      1362000 |         0 | 1362000
   8364 | Completada | 2026-06-02 | 2026-06-07 |      1795000 |    402000 | 2197000
  10534 | Completada | 2026-06-02 | 2026-06-08 |      2526000 |    268000 | 2794000
  19436 | Completada | 2026-06-02 | 2026-06-08 |      2730000 |    162000 | 2892000
   6931 | Completada | 2026-06-03 | 2026-06-10 |      4066000 |    134000 | 4200000
  13688 | Completada | 2026-06-03 | 2026-06-10 |      2565000 |    402000 | 2967000
  16496 | Completada | 2026-06-03 | 2026-06-06 |      2673000 |    402000 | 3075000
  19736 | Completada | 2026-06-03 | 2026-06-09 |      5340000 |    268000 | 5608000
  23105 | Completada | 2026-06-03 | 2026-06-06 |      1578000 |     81000 | 1659000
```



Salida de Script | Tarea terminada en 9,723 segundos

```
  23105 | Completada | 2026-06-03 | 2026-06-06 |      1578000 |     81000 | 1659000
   4585 | Completada | 2026-06-04 | 2026-06-07 |      1545000 |    243000 | 1788000
   8200 | Completada | 2026-06-04 | 2026-06-05 |       310000 |         0 | 310000
   5193 | Completada | 2026-06-05 | 2026-06-10 |       895000 |    134000 | 1029000
   8165 | Completada | 2026-06-05 | 2026-06-09 |      1424000 |    296000 | 1720000
  14640 | Completada | 2026-06-05 | 2026-06-08 |       954000 |    162000 | 1116000
  16725 | Completada | 2026-06-05 | 2026-06-10 |      4080000 |    296000 | 4376000
  23563 | Completada | 2026-06-05 | 2026-06-07 |       442000 |    134000 | 576000
  25340 | Completada | 2026-06-05 | 2026-06-07 |      2730000 |    377000 | 3107000
   3260 | Completada | 2026-06-06 | 2026-06-11 |      2800000 |    268000 | 3068000
  11421 | Completada | 2026-06-06 | 2026-06-08 |      1420000 |    402000 | 1822000
  12760 | Completada | 2026-06-06 | 2026-06-10 |      1084000 |         0 | 1084000
  13336 | Completada | 2026-06-06 | 2026-06-11 |      2200000 |    134000 | 2334000
    170 | Completada | 2026-06-07 | 2026-06-13 |      3156000 |    215000 | 3371000
   4836 | Completada | 2026-06-07 | 2026-06-12 |      1040000 |    243000 | 1283000
   6209 | Completada | 2026-06-07 | 2026-06-08 |       577000 |    377000 | 954000
   7488 | Completada | 2026-06-07 | 2026-06-14 |      1526000 |    243000 | 1769000
   8870 | Completada | 2026-06-07 | 2026-06-11 |      2368000 |    564000 | 2932000
  10065 | Completada | 2026-06-07 | 2026-06-11 |      1816000 |    162000 | 1978000
    489 | Completada | 2026-06-08 | 2026-06-12 |      1820000 |    349000 | 2169000
   2056 | Completada | 2026-06-08 | 2026-06-09 |       576000 |    377000 | 953000
  13285 | Completada | 2026-06-08 | 2026-06-12 |      1340000 |    349000 | 1689000
```



Salida de Script | Tarea terminada en 9,723 segundos

```
   5819 | Completada | 2026-06-26 | 2026-06-30 |      1212000 |    134000 | 1346000
   8076 | Completada | 2026-06-26 | 2026-07-03 |      2226000 |    349000 | 2575000
  15063 | Completada | 2026-06-26 | 2026-06-30 |      5253000 |    511000 | 5764000
  24801 | Completada | 2026-06-26 | 2026-06-28 |       910000 |    402000 | 1312000
   1605 | Completada | 2026-06-27 | 2026-06-30 |      1533000 |    483000 | 2016000
   8210 | Completada | 2026-06-27 | 2026-07-01 |      2104000 |    377000 | 2481000
  21456 | Completada | 2026-06-27 | 2026-07-02 |      1355000 |         0 | 1355000
   2271 | Completada | 2026-06-28 | 2026-06-30 |       414000 |         0 | 414000
  15566 | Completada | 2026-06-28 | 2026-06-29 |       199000 |    483000 | 682000
  20342 | Completada | 2026-06-28 | 2026-07-01 |      1731000 |         0 | 1731000
   8248 | Completada | 2026-06-29 | 2026-07-05 |      3740000 |         0 | 3740000
   9835 | Completada | 2026-06-29 | 2026-06-30 |       735000 |    430000 | 1165000
  22748 | Completada | 2026-06-29 | 2026-07-02 |       657000 |    564000 | 1221000
   1257 | Completada | 2026-06-30 | 2026-07-01 |       735000 |    162000 | 897000
  10390 | Completada | 2026-06-30 | 2026-07-06 |      5610000 |    243000 | 5853000
  16494 | Completada | 2026-06-30 | 2026-07-07 |      1771000 |    483000 | 2254000
  20712 | Completada | 2026-06-30 | 2026-07-01 |       870000 |         0 | 870000
  24986 | Completada | 2026-06-30 | 2026-07-03 |      2415000 |    243000 | 2658000
Reservas liquidadas: 119
Habitaciones: 229531000
Servicios:    31247000
TOTAL A LIQUIDAR: 260778000
```

Fuente: elaboración propia en Oracle SQL Developer.

El procedimiento abre el cursor explícito c_reservas con tres parámetros (alojamiento, inicio y fin del mes) y lo recorre con OPEN, FETCH y CLOSE. Cada reserva se valora noche por noche con fn_valor_estadia, más sus servicios. Solo se liquidan reservas Confirmada y Completada, como pidió la retroalimentación de la Entrega 1.

Figura 15. ***Verificación independiente de la liquidación***

```
394 SELECT COUNT(DISTINCT r.id_reserva) AS reservas,
395        SUM(t.precio_noche)          AS habitaciones
396   FROM reserva r
397   JOIN reserva_habitacion rh ON rh.id_reserva = r.id_reserva
398   JOIN (SELECT LEVEL - 1 AS k FROM dual CONNECT BY LEVEL <= 366) d
399     ON d.k < rh.fecha_checkout - rh.fecha_checkin
400   JOIN temporada te ON rh.fecha_checkin + d.k BETWEEN te.fecha_inicio AND te.fecha_fin
401   JOIN tarifa t     ON t.id_temporada = te.id_temporada AND t.id_habitacion = rh.id_habitacion
402  WHERE r.id_alojamiento = 1
403    AND UPPER(r.estado) IN ('CONFIRMADA', 'COMPLETADA')
```

Salida de Script

Tarea terminada en 0,418 segundos

```
  RESERVAS HABITACIONES
---------- ------------
       119    229531000


 SERVICIOS
----------
  31247000
```

Fuente: elaboración propia en Oracle SQL Developer.

Estas consultas calculan lo mismo que la liquidación, pero sin el cursor y sin la función: expanden las noches y buscan la tarifa directamente. Que los resultados coincidan con la Figura 14 demuestra que el cursor y la función calculan bien.

Figura 16. ***Liquidación masiva de junio de 2026***

```
417 BEGIN
418     pkg_reservas.sp_liquidacion_masiva(2026, 6);
419 END;
420 /
```

Salida de Script

Tarea terminada en 573,999 segundos

```
Liquidacion masiva 2026-06
aloj | nombre                    | reservas | habitaciones |  servicios | total
   1 | Hotel Armenia 1           |      119 |    229531000 |   31247000 | 260778000
   2 | Finca Armenia 2           |        8 |     12321000 |    2441000 | 14762000
   3 | Glamping Armenia 3        |       10 |     25137000 |    4838000 | 29975000
   4 | Hostal Armenia 4          |       59 |     34681000 |   25091000 | 59772000
   5 | Cabanas Armenia 5         |       23 |     69102000 |    8909000 | 78011000
   6 | Hotel Armenia 6           |       97 |    164634000 |   23985000 | 188619000
   7 | Finca Armenia 7           |       17 |     33288000 |    4975000 | 38263000
   8 | Glamping Armenia 8        |       18 |     48190000 |    4262000 | 52452000
   9 | Hostal Armenia 9          |       34 |     19569000 |    7500000 | 27069000
  10 | Cabanas Armenia 10        |       21 |     40347000 |    4965000 | 45312000
  11 | Hotel Armenia 11          |      130 |    239363000 |   14682000 | 254045000
  12 | Finca Armenia 12          |       11 |     16850000 |    2563000 | 19413000
  13 | Glamping Armenia 13       |       11 |     38609000 |    2430000 | 41039000
  14 | Hostal Calarca 14         |       14 |      8281000 |    2159000 | 10440000
  15 | Cabanas Calarca 15        |        6 |     11268000 |    1130000 | 12398000
  16 | Hotel Calarca 16          |       35 |     68385000 |    9553000 | 77938000
  17 | Finca Calarca 17          |        4 |      8970000 |    1053000 | 10023000
  18 | Glamping Calarca 18       |        3 |      9402000 |    1185000 | 10587000
  19 | Hostal Calarca 19         |        4 |      2882000 |    1044000 | 3926000
  20 | Cabanas Montenegro 20     |        7 |     21950000 |    2669000 | 24619000
  21 | Hotel Montenegro 21       |       30 |     68328000 |    9129000 | 77457000
  22 | Finca Montenegro 22       |        6 |      8770000 |    1631000 | 10401000
  23 | Glamping Montenegro 23    |        9 |     16249000 |    1398000 | 17647000
  24 | Hostal Montenegro 24      |       22 |     13105000 |    7199000 | 20304000
  25 | Cabanas Quimbaya 25       |        8 |     24316000 |    1479000 | 25795000
  26 | Hotel Quimbaya 26         |       27 |     51821000 |   11388000 | 63209000
  27 | Finca Quimbaya 27         |        9 |     18395000 |    1479000 | 19874000
  28 | Glamping Quimbaya 28      |        2 |      7234000 |     692000 | 7926000
  29 | Hostal La Tebaida 29      |       14 |      7905000 |    3909000 | 11814000
  30 | Cabanas La Tebaida 30     |        5 |     19117000 |    1829000 | 20946000
  31 | Hotel La Tebaida 31       |       30 |     49093000 |    8667000 | 57760000
  32 | Finca Circasia 32         |        4 |      5829000 |    1594000 | 7423000
  33 | Glamping Circasia 33      |        4 |      4157000 |     707000 | 4864000
  34 | Hostal Circasia 34        |       21 |     11573000 |    4204000 | 15777000
  35 | Cabanas Salento 35        |       14 |     31021000 |    4134000 | 35155000
  36 | Hotel Salento 36          |      112 |    191922000 |   39368000 | 231290000
  37 | Finca Salento 37          |        8 |     18093000 |     928000 | 19021000
  38 | Glamping Salento 38       |       10 |     18780000 |    3116000 | 21896000
```

```
 38 | Glamping Salento 38          |       10 |    18780000 |    3116000 | 21896000
 39 | Hostal Salento 39            |       45 |    28911000 |   10840000 | 39751000
 40 | Cabanas Salento 40           |        4 |    10884000 |    1036000 | 11920000
 41 | Hotel Salento 41             |       82 |   137437000 |   38589000 | 176026000
 42 | Finca Salento 42             |       13 |    18810000 |    5105000 | 23915000
 43 | Glamping Salento 43          |       12 |    14869000 |    1572000 | 16441000
 44 | Hostal Salento 44            |       42 |    25172000 |   11196000 | 36368000
 45 | Cabanas Filandia 45          |        8 |    30473000 |     987000 | 31460000
 46 | Hotel Filandia 46            |       33 |    59198000 |    9769000 | 68967000
 47 | Finca Filandia 47            |        2 |     1816000 |     479000 | 2295000
 48 | Glamping Filandia 48         |       10 |    31380000 |    2907000 | 34287000
 49 | Hostal Filandia 49           |       13 |     8067000 |    3241000 | 11308000
 50 | Cabanas Filandia 50          |        8 |    18105000 |    1878000 | 19983000
 51 | Hotel Filandia 51            |       42 |    60437000 |    7104000 | 67541000
 52 | Finca Filandia 52            |        1 |     2664000 |     160000 | 2824000
 53 | Glamping Buenavista 53       |        5 |     9442000 |     974000 | 10416000
 54 | Hostal Buenavista 54         |       14 |     7376000 |    5215000 | 12591000
 55 | Cabanas Cordoba 55           |       13 |    42243000 |    4347000 | 46590000
 56 | Hotel Cordoba 56             |       31 |    68590000 |   12761000 | 81351000
 57 | Finca Pijao 57               |        5 |    12860000 |    1202000 | 14062000
 58 | Glamping Pijao 58            |        5 |    13078000 |    2011000 | 15089000
 59 | Hostal Genova 59             |        9 |     4996000 |    3072000 | 8068000
```

```
 60 | Cabanas Genova 60            |        8 |    17212000 |    2979000 | 20191000
Alojamientos con reservas: 60 | sin reservas en el mes: 0
Reservas liquidadas: 1371
TOTAL DEL MES: 2679444000

Procedimiento PL/SQL terminado correctamente.
```

Fuente: elaboración propia en Oracle SQL Developer.

El proceso masivo reutiliza el mismo cursor con parámetros para los 60 alojamientos, cambiando solo el alojamiento en cada vuelta. La línea del alojamiento 1 da el mismo total que la Figura 14, lo que confirma que los dos procedimientos comparten un único cálculo.

Figura 17. ***Errores propios de la liquidación***

```
423  BEGIN
424      pkg_reservas.sp_liquidacion_mensual(1, 2026, 13);
425  EXCEPTION WHEN OTHERS THEN
426      DBMS_OUTPUT.PUT_LINE('Mes 13          -> ' || SQLERRM);
427  END;
428  /
429
430  BEGIN
431      pkg_reservas.sp_liquidacion_mensual(999, 2026, 6);
432  EXCEPTION WHEN OTHERS THEN
```

Salida de Script

Tarea terminada en 0,322 segundos

```
Mes 13          -> ORA-20008: El mes debe estar entre 1 y 12

Procedimiento PL/SQL terminado correctamente.
Alojamiento 999 -> ORA-20009: El alojamiento 999 no existe

Procedimiento PL/SQL terminado correctamente.
```

Fuente: elaboración propia en Oracle SQL Developer.

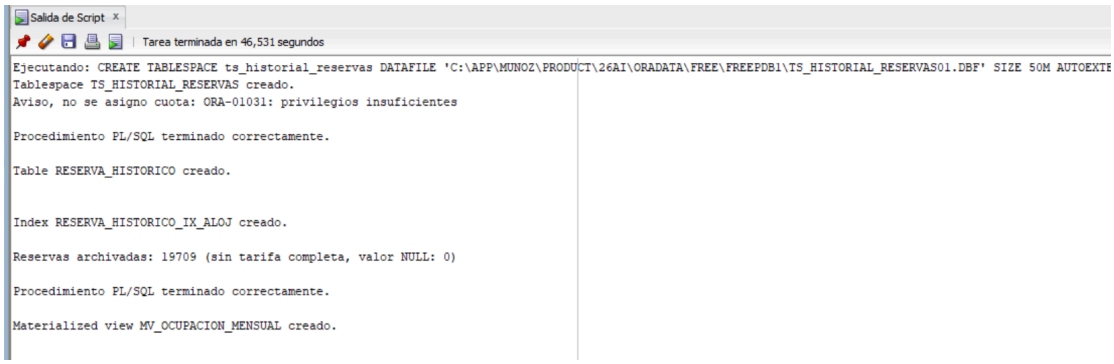
La liquidación valida sus parámetros antes de abrir el cursor y responde con errores propios a un mes inexistente o a un alojamiento que no existe, en lugar de devolver una liquidación vacía que pudiera confundirse con un mes sin ventas.

Prueba 6 · Tablespace del histórico (5 %)

El script 06 crea TS_HISTORIAL_RESERVAS (50 MB, autoextensible hasta 1 GB, gestión local) y guarda allí la tabla RESERVA_HISTORICO con sus dos índices. La ruta del archivo no está escrita a mano: el script la toma de la carpeta de datos de la base, así corre en cualquier computador. El histórico copia las reservas Completada y Cancelada con salida anterior al 1 de julio de 2026. La justificación de separarlo está en Justificación técnica.

Comprobación	Esperado	Obtenido
Ruta del datafile	Calculada por el script	C:\APP\MUNOZ\PRODUCT\26AI\ORADATA\FREE\FREEPDB1\TS_HISTORIAL_RESERVAS01.DBF
Estado del tablespace	ONLINE, LOCAL, AUTO	ONLINE, PERMANENT, LOCAL, AUTO
Segmentos en el tablespace	Tabla y sus 2 índices	RESERVA_HISTORICO (2.048 KB), RESERVA_HISTORICO_PK (704 KB) y RESERVA_HISTORICO_IX_ALOJ (768 KB)
archivadas vs en_reserva	Iguales	19.709 = 19.709, del 01/01/2024 al 30/06/2026
sin_valor	0	0

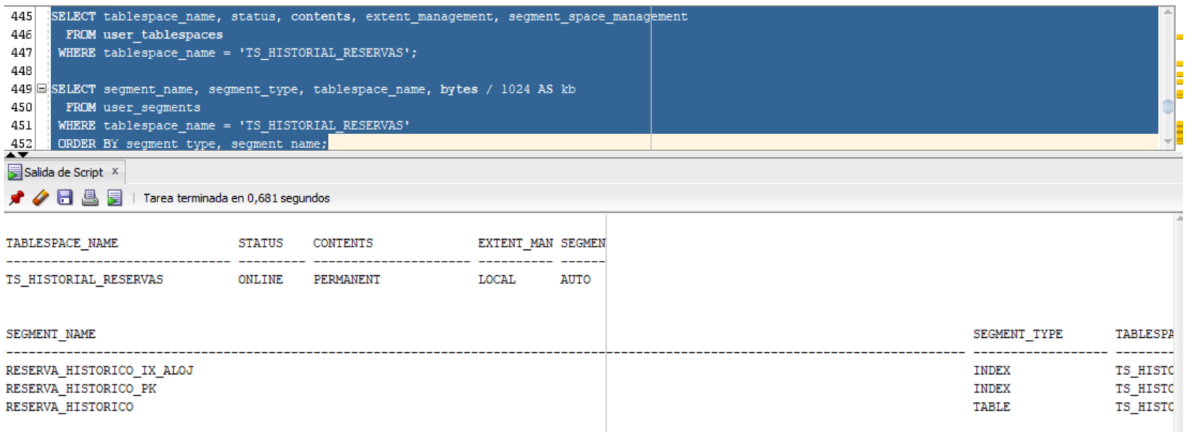
Figura 18. ***Creación del tablespace y carga del histórico***



Fuente: elaboración propia en Oracle SQL Developer.

El script calcula la carpeta del datafile a partir de la base en lugar de escribir una ruta fija, para que corra en cualquier computador. El histórico archivó 19.709 reservas cerradas sin ningún valor nulo. El aviso de cuota es esperado: el usuario ya tiene UNLIMITED TABLESPACE y el script lo trata como aviso, no como error.

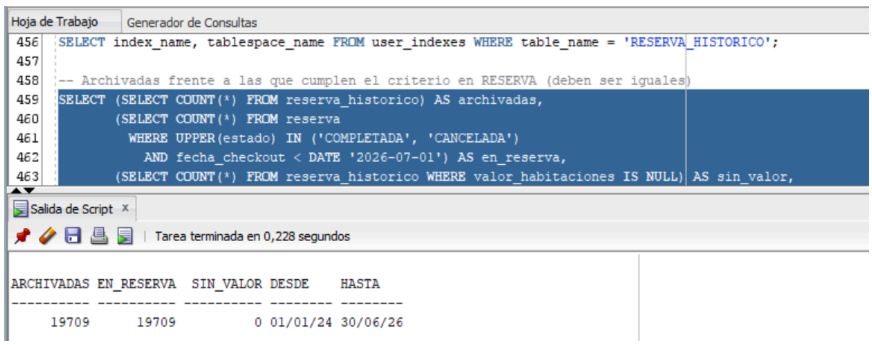
Figura 19. ***Tablespace TS_HISTORIAL_RESERVAS y sus segmentos***



Fuente: elaboración propia en Oracle SQL Developer.

El tablespace propio está en línea, con gestión local de extensiones y de espacio. La tabla del histórico y sus dos índices quedaron allí, separados de las tablas operativas, por las razones que se explican en la Justificación técnica.

Figura 20. ***Contenido del histórico de reservas***



Fuente: elaboración propia en Oracle SQL Developer.

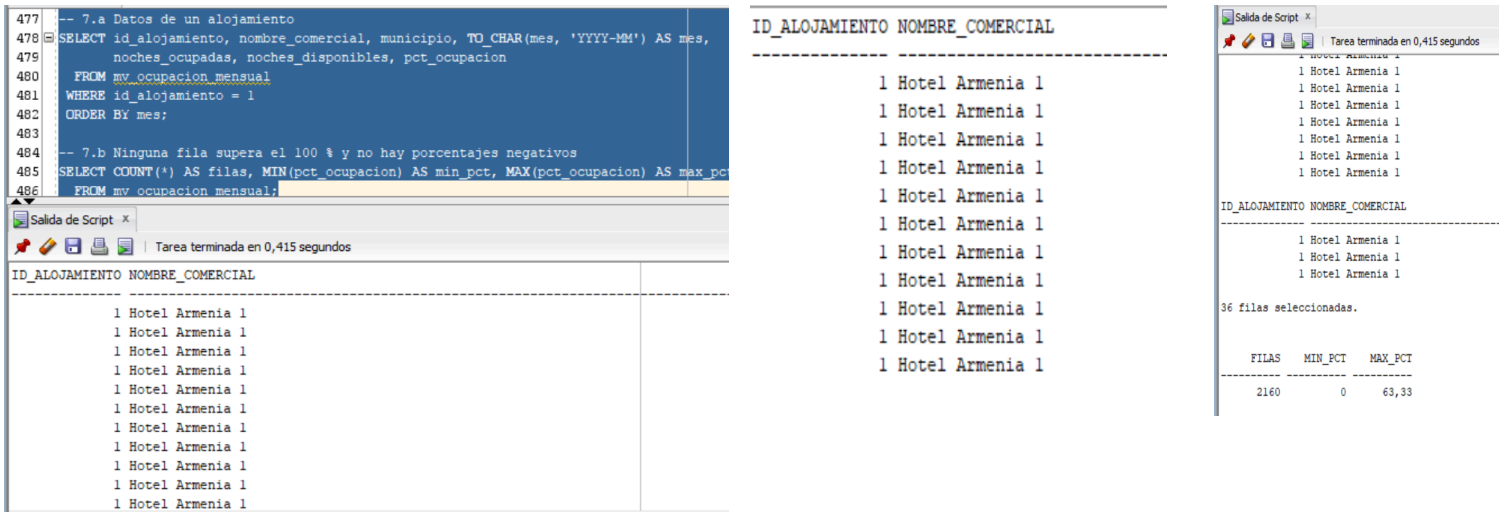
Las reservas archivadas coinciden con las que cumplen el criterio: cerradas y con salida antes del 1 de julio de 2026. El histórico es una copia; las reservas siguen en RESERVA porque PAGO, RESENA y RESERVA_SERVICIO las referencian.

Prueba 7 · Vista materializada de ocupación mensual (15 %)

MV_OCUPACION_MENSUAL guarda, por alojamiento y mes, las noches ocupadas, las noches disponibles (habitaciones × días del mes) y el porcentaje de ocupación. Cuenta solo reservas Confirmada y Completada y cada habitación-noche una sola vez. Se refresca completa todos los días a las 2:00 a. m. y a mano con DBMS_MVIEW.REFRESH; la justificación está en Justificación técnica.

Comprobación	Esperado	Obtenido
Filas de la vista	Alojamientos × meses	2.160 = 60 × 36 meses (ene 2024 a dic 2026)
min_pct / max_pct	≥ 0 / ≤ 100	0 / 63,33
Junio 2026, alojamiento 1: noches_a_mano vs noches_vista	Iguales	604 = 604 (55,93 %)
Enero 2027 antes del refresco (paso B)	Sin filas	Sin filas
Enero 2027 después del refresco (paso D)	3 noches ocupadas	3 de 1.116 disponibles (0,27 %)
Limpieza (paso E)	1 reserva borrada	1 reserva borrada
refresh_method y próxima ejecución	COMPLETE, mañana a las 2:00 a. m.	COMPLETE; próxima: 30/10/2026 (TRUNC(SYSDATE) + 1 + 2/24)

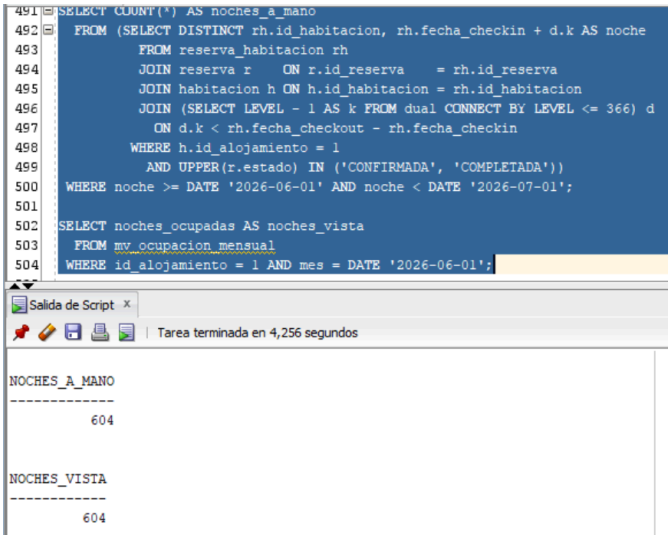
Figura 21. ***Ocupación mensual del alojamiento 1 en la vista materializada***



Fuente: elaboración propia en Oracle SQL Developer.

La vista guarda ya calculada la ocupación de cada alojamiento por mes. Muestra los 36 meses de 2024 a 2026, incluidos los de 0 %, y ningún mes pasa del 100 % porque cada habitación-noche se cuenta una sola vez.

Figura 22. ***Comprobación de la vista contra el cálculo a mano***



Fuente: elaboración propia en Oracle SQL Developer.

Un cálculo hecho a mano, por fuera de la vista, da las mismas 604 noches ocupadas en junio de 2026. Esto demuestra que la vista precalcula la ocupación correctamente.

Figura 23. ***Ocupación por municipio y mes leída de la vista***

```
SELECT municipio, TO_CHAR(mes, 'YYYY-MM') AS mes,
       SUM(noches_ocupadas) AS ocupadas, SUM(noches_disponibles) AS disponibles,
       ROUND(100 * SUM(noches_ocupadas) / SUM(noches_disponibles), 2) AS pct
  FROM mv_ocupacion_mensual
```

Salida de Script | Resultado de la Consulta

SQL | Todas las Filas Recuperadas: 432 en 0,177 segundos

	MUNICIPIO	MES	OCUPADAS	DISPONIBLES	PCT
1	Armenia	2024-01	350	5301	6,6
2	Armenia	2024-02	401	4959	8,09
3	Armenia	2024-03	643	5301	12,13
4	Armenia	2024-04	771	5130	15,03
5	Armenia	2024-05	864	5301	16,3
6	Armenia	2024-06	1766	5130	34,42
7	Armenia	2024-07	2088	5301	39,39
8	Armenia	2024-08	1006	5301	18,98
9	Armenia	2024-09	880	5130	17,15
10	Armenia	2024-10	422	5301	7,96
11	Armenia	2024-11	359	5130	7
12	Armenia	2024-12	1889	5301	35,63
13	Armenia	2025-01	1599	5301	30,16
14	Armenia	2025-02	465	4788	9,71
15	Armenia	2025-03	457	5301	8,62

SQL | Todas las Filas Recuperadas: 432 en 0,177 segundos

	MUNICIPIO	MES	OCUPADAS	DISPONIBLES	PCT
415	Salento	2025-07	1437	3255	44,15
416	Salento	2025-08	880	3255	27,04
417	Salento	2025-09	812	3150	25,78
418	Salento	2025-10	351	3255	10,78
419	Salento	2025-11	369	3150	11,71
420	Salento	2025-12	1598	3255	49,09
421	Salento	2026-01	1192	3255	36,62
422	Salento	2026-02	462	2940	15,71
423	Salento	2026-03	609	3255	18,71
424	Salento	2026-04	1170	3150	37,14
425	Salento	2026-05	947	3255	29,09
426	Salento	2026-06	1722	3150	54,67
427	Salento	2026-07	1807	3255	55,51
428	Salento	2026-08	1031	3255	31,67
429	Salento	2026-09	1054	3150	33,46
430	Salento	2026-10	864	3255	26,54
431	Salento	2026-11	216	3150	6,86
432	Salento	2026-12	1119	3255	34,38

Fuente: elaboración propia en Oracle SQL Developer.

El reporte de ocupación por municipio y mes que necesita la gerencia se obtiene leyendo la vista, sin volver a expandir las reservas en noches. Ese es el beneficio de tenerla precalculada.

Figura 24. ***Vista materializada antes del refresco***

```
    v_res reserva.id_reserva%TYPE;
BEGIN
    SELECT MIN(id_habitacion) INTO v_hab FROM habitacion WHERE id_alojamiento = 1;
    pkg_reservas.sp_crear_reserva(1, v_hab, DATE '2027-01-05', DATE '2027-01-08', 1, v_res);
    UPDATE reserva SET estado = 'Confirmada' WHERE id_reserva = v_res;
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Reserva de prueba ' || v_res || ' confirmada');
END;
/

-- Paso B: la vista todavia no la ve (sin filas para enero de 2027)
SELECT TO_CHAR(mes, 'YYYY-MM') AS mes, noches_ocupadas
```

Salida de Script | Tarea terminada en 0,646 segundos

```
Reserva 25501 creada. Valor de la estadia: 1011000
Reserva de prueba 25501 confirmada

Procedimiento PL/SQL terminado correctamente.
no se ha seleccionado ninguna fila
```

Fuente: elaboración propia en Oracle SQL Developer.

Se creó y confirmó una reserva nueva, pero la vista todavía no la muestra. Esto evidencia que la vista guarda una foto de los datos y solo cambia cuando se refresca.

Figura 25. ***Vista materializada después del refresco manual***

```
537  BEGIN
538      DBMS_MVIEW.REFRESH('MV_OCUPACION_MENSUAL', 'C');
539  END;
540  /
541
```

Salida de Script | Tarea terminada en 4,42 segundos

```
Procedimiento PL/SQL terminado correctamente.

MES     NOCHES_OCUPADAS NOCHES_DISPONIBLES PCT_OCUPACION
------- --------------- ------------------ -------------
2027-01               3               1116 ,27
```

Fuente: elaboración propia en Oracle SQL Developer.

Después del refresco manual con DBMS_MVIEW.REFRESH aparecen las 3 noches nuevas. Así, aunque el refresco automático es diario, el dato se puede actualizar al instante cuando se necesite.

Figura 26. ***Política de refresco programada de la vista***

```
552 BEGIN
553     DELETE FROM reserva_habitacion
554      WHERE id_reserva IN (SELECT id_reserva FROM reserva
555                           WHERE id_cliente = 1 AND id_alojamiento = 1
556                             AND fecha_checkin  = DATE '2027-01-05'
557                             AND fecha_checkout = DATE '2027-01-08');
558     DELETE FROM reserva
```

Salida de Script | Tarea terminada en 5,227 segundos

```
Reservas de prueba borradas: 1

Procedimiento PL/SQL terminado correctamente.

MVIEW_NAME                                                                      REFRESH_M REFRESH_ LAST_REF
------------------------------------------------------------------------------- --------- -------- --------
MV_OCUPACION_MENSUAL                                                            DEMAND    COMPLETE 29/10/26


RNAME                                                                           NEXT_DAT INTERVAL
------------------------------------------------------------------------------- -------- ------------------
MV_OCUPACION_MENSUAL                                                            30/10/26 TRUNC(SYSDATE) + 1
```

Fuente: elaboración propia en Oracle SQL Developer.

La vista se recalcula completa (COMPLETE) y no en cada COMMIT (DEMAND), con un trabajo programado todos los días a las 2:00 a. m. Se eligió así porque es un reporte mensual que tolera un día de atraso, y recalcularlo en cada reserva haría lenta la operación (ver Justificación técnica). La reserva de prueba se borró para dejar la base como estaba.

Justificación técnica

fn_valor_estadia

El precio no se calcula como tarifa × noches porque una estadía puede cruzar varias temporadas (sección 1.2 del enunciado). La función genera una fila por noche con CONNECT BY, la cruza con TEMPORADA por rango de fechas y con TARIFA por habitación y temporada, y suma. El día de salida no se cobra: una noche va del check-in incluido al check-out excluido. Prefiere fallar a devolver un valor equivocado: -20002 si una noche no tiene tarifa y -20012 si una noche cae en dos temporadas.

sp_crear_reserva

Las validaciones van de la más barata a la más costosa (fechas y huéspedes antes de consultar tablas) y todas ocurren antes del primer INSERT, así una reserva inválida nunca se escribe a medias. Una reserva Pendiente también bloquea la habitación; si no, dos clientes podrían apartarla al tiempo mientras pagan. El procedimiento no hace COMMIT para que pueda formar parte de una transacción mayor, como la de reserva y pago de la Entrega 3. Los identificadores se calculan con MAX + 1 porque el DDL no definió secuencias; el manejo de dos sesiones simultáneas se resuelve en la Entrega 3 con SELECT … FOR UPDATE.

Cursor de liquidación

Es un cursor explícito con tres parámetros (alojamiento, inicio del mes, inicio del mes siguiente). Los parámetros permiten reutilizar la misma consulta para cualquier alojamiento y mes sin concatenar SQL. Está declarado en el cuerpo del paquete para que la liquidación individual y la masiva usen exactamente el mismo cálculo. Cuenta solo reservas Confirmada y Completada, como pidió la retroalimentación de la Entrega 1, y cada reserva se liquida en el mes de su check-in. Si algo falla a mitad del recorrido, el manejador de excepciones cierra el cursor para que la siguiente llamada no dé ORA-06511.

Paquete

PKG_RESERVAS separa lo público (especificación) de lo interno (cuerpo). Lo que solo usan las liquidaciones, el cursor y la validación del mes, queda privado. Todos los errores propios tienen un código fijo:

Código	Significado	Lo lanza
-20001	Salida no posterior a la entrada	Función y procedimiento
-20002	Noches sin tarifa	Función
-20003	Habitación inexistente	Función y procedimiento
-20004	Cliente inexistente	Procedimiento
-20005	Capacidad excedida	Procedimiento
-20006	Habitación no disponible	Procedimiento
-20007	Huéspedes inválidos	Procedimiento
-20008	Año o mes inválido	Liquidaciones
-20009	Alojamiento inexistente	Liquidación mensual
-20010	Solape de reservas	TRG_RH_SIN_SOLAPE
-20011	Fechas de la línea inválidas o fuera de la reserva	TRG_RH_SIN_SOLAPE
-20012	Noche con dos temporadas o tarifa repetida	Función

Disparadores compuestos

Un disparador de sentencia puro no tiene :OLD ni :NEW, así que no puede registrar el precio anterior y el nuevo que pide la auditoría. Un disparador de fila que consulta su propia tabla da ORA-04091 (tabla mutante) en un UPDATE o en un INSERT de varias filas. El compuesto resuelve ambos casos. En la auditoría, cada fila se anota y la escritura ocurre una vez por sentencia. En el solape, cada fila se valida en BEFORE EACH ROW y la comparación contra la tabla se hace en AFTER STATEMENT. Las pruebas 3c y 3d lo demuestran. El disparador de solape protege la tabla aunque alguien inserte sin pasar por el procedimiento.

Requisito del enunciado	Disparador	Sección que lo cumple
Disparador de sentencia: auditoría de TARIFA (quién, cuándo, valor anterior y nuevo)	TRG_TARIFA_AUDITORIA	AFTER STATEMENT: escribe la auditoría una sola vez por sentencia, con SESSION_USER y SYSDATE
Disparador de fila: impedir que una reserva se solape con otra de la misma habitación	TRG_RH_SIN_SOLAPE	BEFORE EACH ROW: valida cada fila insertada o modificada antes de que entre a la tabla

Tablespace del histórico

Las reservas cerradas no se modifican, solo se consultan, y crecen con cada temporada. Separarlas en TS_HISTORIAL_RESERVAS tiene cuatro ventajas. El histórico crece en su propio archivo sin quitar espacio a las tablas operativas. Se respalda con otra frecuencia que lo operativo. Puede pasar a solo lectura (ALTER TABLESPACE … READ ONLY) y moverse a un disco más barato. Si se llena o se daña, las reservas del día siguen funcionando.

Configuración: 50 MB iniciales con crecimiento de 10 MB hasta 1 GB, gestión local de extensiones y de espacio en segmentos. La ruta del archivo se calcula al crear el tablespace para que el script corra en cualquier computador. La tabla guarda una copia y no mueve las reservas, porque PAGO, RESENA y RESERVA_SERVICIO las referencian.

Vista materializada y su política de refresco

La ocupación mensual expande cada reserva en sus noches y agrupa por alojamiento y mes, un cálculo costoso que la gerencia consulta muchas veces. La vista lo guarda ya hecho. Cuenta cada habitación-noche una sola vez y muestra con 0 % los meses sin ventas.

Resumen de resultados

Componente	Peso	Prueba	Casos	Figuras	¿Pasó?
fn_valor_estadia	20 %	1	5	5 a 7	Sí, 5 de 5
sp_crear_reserva	15 %	2	9	8 y 9	Sí, 9 de 9
Cursor de liquidación	15 %	5	6	14 a 17	Sí, 6 de 6
Paquete	10 %	0	4	1 a 4	Sí, 4 de 4
Disparador de sentencia	10 %	4	5	12 y 13	Sí, 5 de 5
Disparador de fila	10 %	3	7	10 y 11	Sí, 7 de 7
Tablespace	5 %	6	4	18 a 20	Sí, 4 de 4
Vista materializada	15 %	7	6	21 a 26	Sí, 6 de 6

VIGILADA MINEDUCACIÓN



Base de Datos II
Carolina Londoño Idarraga
Armenia, Quindío – Colombia
2026-2

UNIQUINDÍO, en conexión territorial
Carrera 15 Calle 12 Norte Tel: (606) 7 35 93 00 Armenia - Quindío - Colombia
www.uniquindio.edu.co